Performing Fast COUNT() Queries

COUNT(*) queries are known to be slow, as explained in *Faster PostgreSQL Counting*.[20] They scan the whole table for accurate results. Counts might be for all rows in a table or for a filtered set of rows.

You can achieve fast counts for both unfiltered and filtered scenarios, but you'll need to use some tricks.

The easiest trick is to use the count estimates that PostgreSQL tracks for each table instead of running a COUNT() query. These estimates will be off a bit but are incredibly quick to fetch.

Open up psql from your terminal for the Rideshare database. Run the following SQL statements. Since this is counting all tables named users, drop the temp.users table if you still have it, by running DROP TABLE temp.users CASCADE; to get a single count estimate.

sql/estimated_count_reltuples.sql

```
-- Update stats
ANALYZE users;

-- Use reltuples for estimate
SELECT reltuples::NUMERIC FROM pg_class WHERE relname='users';
```

The first statement updates the statistics for the users table and the second statement fetches the estimated row count.

The fast_count[21] gem has been added to Rideshare, which enables faster counting for Active Record. The gem adds a Migration that adds a database function for fast counts.

For Rideshare, run bin/rails console and try out the User.fast_count method. Compare it with User.count for speed and accuracy.

```
User.fast_count
```

Although you haven't yet worked in detail with Query execution plans, the execution plan text itself can be used as a data source.

This trick uses the query execution plan to get fast estimated filtered counts. Try running bin/rails console in the Rideshare directory and comparing these two Active Record snippets that count filtered sets of Users. If you ran the data generator you should have 10 million users.

```
User.where("last_name ILIKE 'lname123%'").count
```

20. https://www.citusdata.com/blog/2016/10/12/count-performance/
21. https://github.com/fatkodima/fast_count